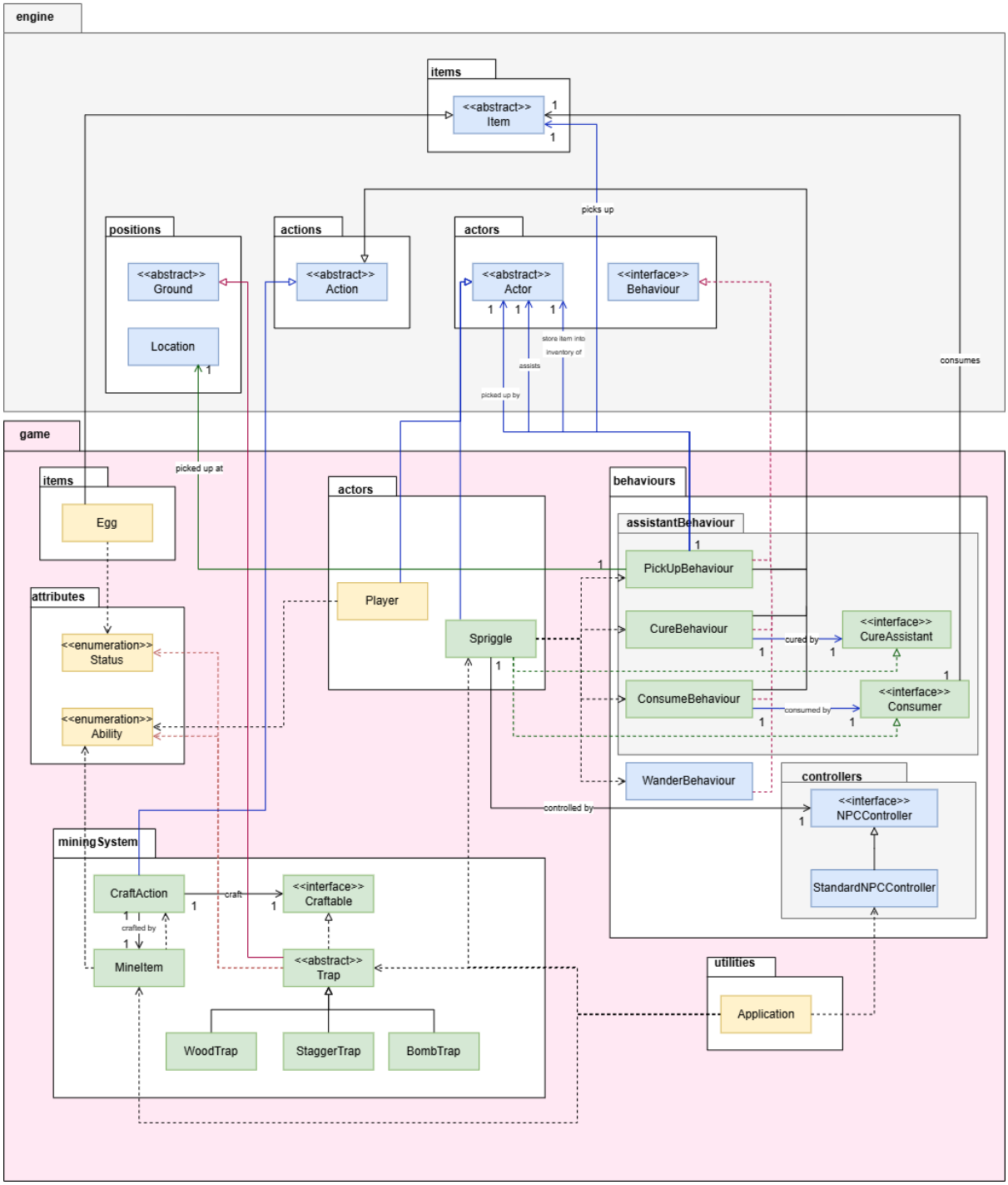
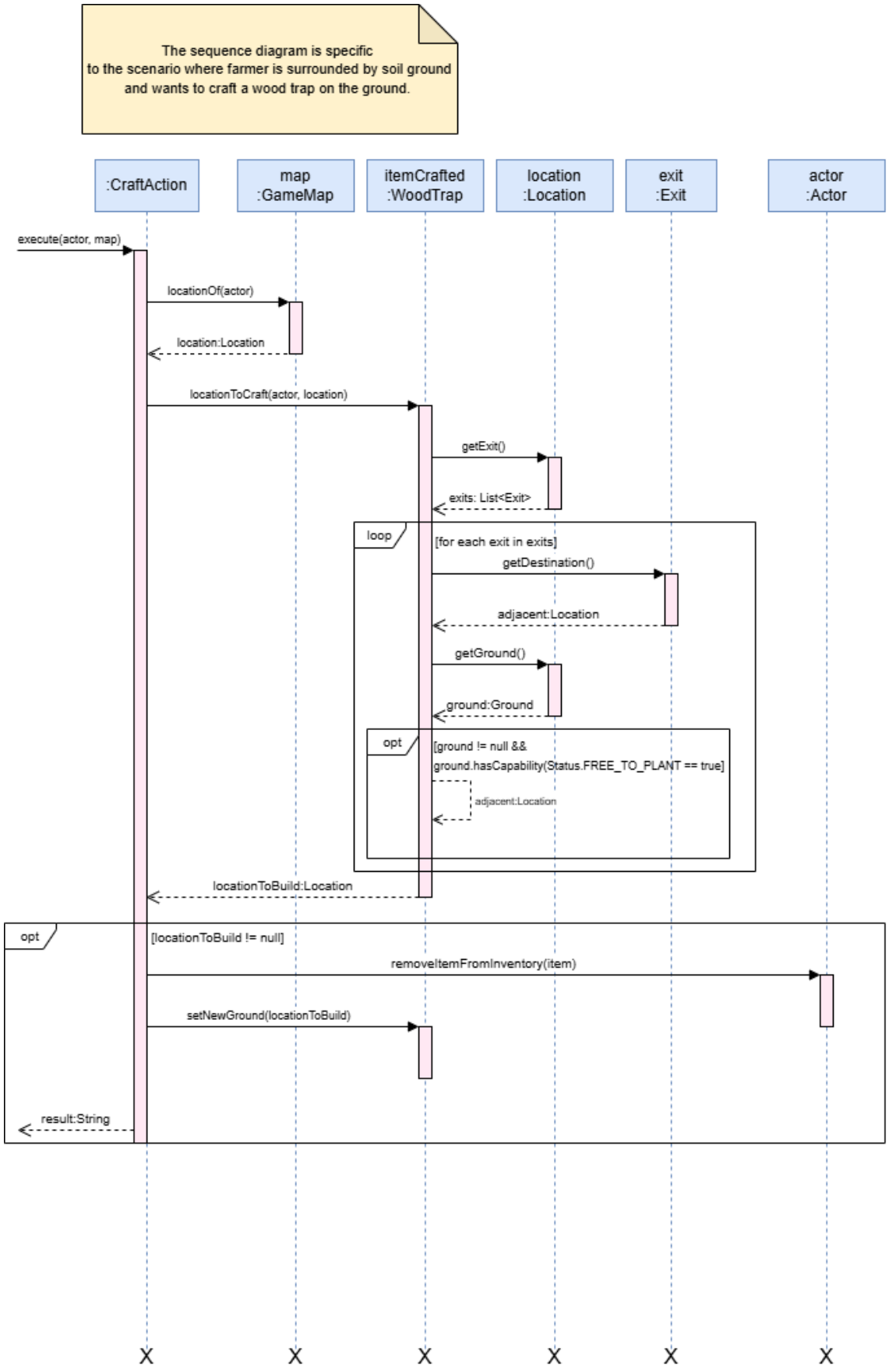
Design Documentation REQ 4

UML Diagram

Class indicator	
Yellow	Modified class
Blue	Existing class
Green	New class



Sequence Diagram



Design Rationale

Design 1

Create Consumer interface to represent an NPC can consume item.

Pros	Cons
Ensure implementers personalise own consuming logic from the methods defined in this interface, enforce its interchangeability without breaking functionality. (LSP)	itemToConsume() defined in this interface restricted NPC to only consume item type but not an actor.
Other NPCs which can also be a consumer in the future can be easily extended by implementing this interface. (OCP)	
Focus on handling consuming logic only by avoiding unnecessary methods. (ISP)	

Design 2

Create CureAssistant interface to represent an NPC can cure item/creature.

Pros	Cons
Reduce connascence of meaning by defining condition, effect and cost checker separately into different methods.	Implementers are assumed to have curing cost after curing due to actorConsciousness(). This may lead empty logic/unnecessary method in some cases. (violate ISP)
Ensure implementers always personalise own curing condition, effect and cost. (LSP)	

Alternative approach:

Remove actorConsciousness() defined in this interface and directly check in cureEffect() by changing into boolean return data type.

- Pros: other implementers without have any curing cost does not need to leave this redundant method empty, follow ISP.
- Cons: increase connascence of meaning by handling multiple logic in single method. In addition, despite not having curing cost, cureEffect() always need to

return true as well, which doesn't make it a better approach as compared to current implementation.

Design 3

Create ConsumeBehaviour to allow NPC to perform auto-consuming action when a certain condition is met.

Create CureBehaviour to allow NPC to perform auto-curing action when a certain condition is met.

Create PickUpBehaviour to allow NPC to pick up an item at the surrounding and add to own/farmer inventory based on hitpoints.

These classes extend from Action and implement Behaviour class to override execution, aiming to allow targeted NPC to perform these behaviours based on personalised conditions.

Pros	**Cons**
Encapsulate respective behaviour on respective behavioural class, separating consuming, pick up and curing logic into different classes. (SRP)	Handle action and behaviour logic in single class. (violate SRP)
Allows NPC to have more flexibility in controlling allowable behaviour. i.e. NPC may be able to consume but unable to cure others. (OCP)	PickUpBehaviour have lower flexibility on picking up item as it always will store all items in farmer inventory when implementer hitpoint > 50%.
Allows adding these behaviours without knowing its implementation details, promoting information hiding.	

Alternative approach:

Remove inheritance towards Action class and create new NPCConsumeAction, NPCCureAction and NPCPickUpAction to be the subclass of Action class.

- Pros: Cleaner code and better logic separation between Action and Behaviour.
- Cons: May overengineered or overcomplex the requirement.

Reason of designing PickUpBehaviour logic into decision making based on hitpoints:

- Assistant prioritizes on assisting farmer at its fullest. If it can still survive, we will not let item to be consumed by assistant first then only farmer, since it's unreasonable for assistant restoring own hitpoint while farmer is suffering from hunger.

- Ensure farmer get item needed immediately.

Design 5

Create Spriggle class to represent a NPC act as farmer's assistant. This class extends from NPC class and implements Consumer, CureAssistant interface.

Pros	**Cons**
Encapsulate personalised condition, behaviours, and effect from interfaces implemented, separate from other NPCs. (SRP)	
New assisting capabilities can be easily extended by implementing new interface in this class and override its method without modifying other classes. (OCP)	

Design 6

Create MineItem class to represent a mining material on the map which can be picked up by an actor. i.e. Spriggle.

This class have association with Craftable interface to get the Trap instance created from this material during instantiation.

Pros	**Cons**
New MineItem type introduced only requires passing its name, crafted ground item as argument, promoting scalability and maintainability. Reduce number of classes needed.	Tight coupling to Craftable interface. i.e. when a ground item is unknown during instantiation, this class may require modification.
Safely substitutable for Item because it overrides behaviour without changing expected contracts. (LSP)	

Design 7

Create Trap abstract class to represent a trap crafted from MineItem. This class extends from Ground class and implements Craftable interface.

Pros	Cons
Depends on abstraction – Ground abstract class and Craftable interface but not concrete classes, avoid downcasting and using instanceof. (DIP)	Subclasses must be able to implement both abstract method and interface method without leaving them empty.
WoodTrap, StaggerTrap and BombTrap inherits common logic without redefining it multiple times, avoiding code duplication. (DRY)	
More traps can be introduced by extending from this class and define their respective effect to enemies. (OCP) i.e. HallucinationTrap which causes enemies at the surrounding to hallucinate.	

Design 8

Create WoodTrap, StaggerTrap, and BombTrap subclasses extend from Trap abstract class. Each class represents different trap can be built on different ground with different mining materials and effect on the map.

Pros	Cons
Encapsulate trap logic for each trap type into its own class, avoiding exposure of internal trap condition and effects. (SRP)	Current location choice is limited, may result in duplicated location-checking logic in the future. (violate DRY)
Whenever Trap instance is expected, these three subclasses are substitutable without causing any error. (LSP)	

Alternative approach:

Extract location-checking logic into a shared utility class/helper class.

- Pros: Solve potential DRY violation issue when trap is extended.

- Cons: Less flexibility if traps need very different placement rules.

Keeping current design to maintain SRP and keep trap behaviour self-contained but not shared logic to external classes.

Design 9

Create Craftable interface to represent an item is craftable on the ground. i.e. Trap can be crafted on the ground if a mining material is picked up.

Pros	**Cons**
Implementers only need to override methods required for new craftable ground, avoiding unnecessary methods. (ISP)	
Allows CraftAction to enforce its action can only be carry out by implementers.	

Design 10

Create CraftAction class to represent an action to craft an item to the ground when user chose this action on menu. This class extends from Action class and has association with MineItem and Craftable interface.

Pros	**Cons**
Encapsulate crafting functionality only, separating from other action. (SRP)	Tight coupling to Craftable ground item. When it's unknown during instantiation, this class will require modification.
Simply by adding this instance to MineItem class, farmer can perform it without knowing its implementation details, promoting information hiding.	
Depends on abstraction – Action abstract class and Craftable interface but not concrete classes, avoid downcasting. (DIP)	

Design 11

Add CRAFTING_ABILITY enum into Ability enum class. This enum is used in Player class to represent farmer has the capability to craft a mining material into trap.

Pros	Cons
Avoid using instanceof to check if an actor belong to Player class.	
Maintain consistency.	

Drawback of REQ 4 Designs:

- They may assist farmer, but they will not apply the same effect as farmer.
    - Consume effect
    - Not all farmer edible item will be pet edible
- One assistant will not be able to assist in curing all curable target.
    - Current assist only in curing cursed ground
- Trap need to be first created and pass as dependency injection during MineItem instantiation.
    - In simple word, when Spriggle pick up the mining material on the ground, farmer inventory eventually already stored not only the Wood but the Trap as well. Farmer will decide whether to craft it at which ground.

Potential alternatives and designs:

- Let one assistant can assist farmer wholly
    - Pros: assist farmer at its fullest
    - Complex and more classes needed (beyond requirement specification)
- Let player to decide which trap to create based on farmer inventory.
    - We do not need to create the Trap when instantiating MineItem in Application class.
    - i.e. we check whether farmer inventory contains Wood, if yes then we create it.
    - Approach: Add CRAFTABLE_WOOD enum class into Status class, add this capability to WoodTrap. Then, create another WoodCraftable class by supplying the trap can be crafted when farmer inventory contains craftable wood. The same concept applies to other mining materials and traps.
    - Pros: more flexible since now player can decide himself what trap to create from the wood picked up.
    - Cons: More complex design (beyond requirement specification)

DRY, SOLID, connascence principle

Every class only focuses on one responsibility. Consumer interface focuses on NPC consuming logic; CureAssistant interface focuses on NPC curing logic; ConsumeBehaviour, CureBehaviour, PickUpBehaviour classes each handle respective auto-behaviour, separating each other into independent classes; Spriggle class encapsulate all personalised conditions, effects, and behaviours from methods defined from interfaces; MineItem focuses on mining material allowable actions and handle own attributes; Trap abstract class defines common logic shared across all of its subclasses (WoodTrap, StaggerTrap, BombTrap); Craftable interface focuses on crafting logic and CraftAction class focuses on executing crafting/building logic when action is invoked. These designs show **Single Responsibility Principle (SRP)** is achieved. However, extra concern is needed for behavioural classes as they handle both action and behaviour logic in a single class which may violate this principle when the complexity increases in the future.

When an NPC can assist farmer like what Spriggle does, it can be easily extended by implementing respective capabilities interface, which are Consumer or CureAssistant Interfaces without modifying existing classes. Interfaces are created to be applicable to every cases. Besides, when we have more mining material introduced, we may also consider introducing more trap for farmer to craft on the ground. These traps can simply extend from Trap class and personalise own crafting effect and condition without exposing to other classes as well, simultaneously promoting information hiding. These designs certainly avoid using if-else statements or instanceof() to check if an entity is an instance with consuming, curing capabilities, supporting **the Open-Close Principle (OCP)** by being open to extension but close to modification. However, MineItem and CraftAction classes which have tight coupling to Craftable may violate this principle when design become more complex in the future. Since CraftAction only invoke methods from Craftable, when more complex logic is introduced, this interface may require modification.

Creating interfaces such as Craftable, Consumer and CureAssistant ensure implementers to personalise own behaviour, condition and logic in the valley, simultaneously enforce its consistency and interchangeability without breaking functionalities. The abstract method, craftEffect() defined in Trap class ensures its subclasses always define this method for all crafting effect but not using other method names. In other words, wherever Trap instance is expected, its subclasses can be substituted without breaking the behaviour of the program. It maintains the contract of its parent class. These designs prevent inconsistent logic, which achieve **Liskov Substitution Principle (LSP).**

Craftable interface has only locationToCraft() and setNewGround() methods, focuses on crafting a trap onto the ground; Consumer() interface contains canConsume(),

consume(), and itemToConsume(), focuses entirely on pre-condition, instant effect and post-condition of consuming logic. These designs avoid unnecessary methods, keeping them focused and minimal. Unrelated entity instances do not need to implement these interfaces, preventing unused methods or forced dependencies. This means that **Interface Segregation Principle (ISP)** is followed. Nevertheless, extra concern is needed in CureAssistant interface as actorConsciousness() are forcing all curing will consume hitpoint/stamina even if not application, otherwise this checker potentially to be redundant, which may lead to empty logic, and violate ISP.

ConsumeBehaviour, CureBehaviour and PickUpBehaviour classes depends on abstraction – extends from Action abstract class, implements Behaviour interface rather than concrete classes like Spriggle. Additionally, Trap abstract class depends on Ground abstract class and implements Craftable interface, as well as CraftAction class which depends on Action abstract class and have association with Craftable rather than concrete classes like WoodTrap, StaggerTrap and BombTrap. This allows adding new trap or farmer assistant instanced easily without causing heavy dependency on lower modules, avoid downcasting and violation of OCP simultaneously. This means that **Dependency Inversion Principle (DIP)** is followed.

CraftAction extending from Action allow crafting logic to be centralized to one method (execute()), changes to crafting logic only require updating this method, reducing redundancy in defining the same action in all classes. The same concept applies to behavioural class as well. Any NPC which can be farmer's assistant will be able to reuse these behavioural classes simply by calling addBehaviour(). Besides, Trap subclasses can inherit all common logic defined in abstract class, which avoid repeating the same code in every class. These practices certainly avoid code duplication and follow OOP concept. **(DRY)**

To strength code maintainability and extensibility, connascence is reduced across the designs. **Connascence of meaning** is reduced by centralizing logic for determining who is affected by a trap (Status.HOSTILE_TO_ENEMY) in a Trap abstract class. This shared meaning is enforced through canActorEnter(), ensuring consistent behaviour across all subclasses without redundant condition checks. Besides, relying on abstract type – Craftable reduces misuse and improve clarity (**Connascence of Type**). Behavioural methods follow consistent execution patterns where only effect-specific logic needs to be defined (**Connascence of Execution**), ensuring low coupling with the game loop itself. Common method names like consume() are abstracted via shared Consumer interface **(Connascence of Name),** allowing polymorphic use without knowledge of concrete classes. Parameter clarity and helper methods reduces algorithmic connascence as well. These design choices ensure the system remains flexible, readable and easy to extend.